DODO PAYMENTS

Backend Engineering Take-Home

Invoice & Payment Service

Mid-Level Engineer · Backend Hiring

At a Glance

Time budget	4 to 6 hours of focused work. Stop and document remaining work if you exceed.
Language	Rust strongly preferred (Axum, Actix, or Warp). Other languages allowed with a brief justification in the README.
Database	PostgreSQL with migrations.
Setup	`docker compose up` runs the app, database, and mock PSP with no manual steps.
AI tools	Allowed. Disclosure of usage is mandatory and graded.
Primary deliverable	**`DESIGN.md`** is the design document we evaluate most heavily.
Demo	5 to 10 minute Loom (or equivalent) video walkthrough is required.

On AI Tools (Read This First)

You may use AI assistants such as Copilot, Cursor, Claude, or ChatGPT freely. We use them too. What we evaluate is your judgment, not your typing speed.

> **Mandatory deliverable: AI_USAGE.md**
>
> A short, honest, specific document describing how AI was used. A blank or generic file is a strong negative signal. A thoughtful one, even if short, is a strong positive signal. This document is graded.

AI_USAGE.md must contain

1. **Which AI tools you used and for what.** Be specific. Examples:
   - Cursor autocomplete for handler boilerplate.
   - Claude to draft the initial DB schema, which I revised because [reason].
   - ChatGPT to explain Postgres advisory locks. I ended up not using them because [reason].
2. **Three decisions you made yourself,** against or independent of AI suggestions. For each: what the AI proposed (if anything), what you chose, and why.

3. **One thing the AI got wrong** or that you had to correct. If you genuinely had nothing to correct, say so and explain how you verified correctness.

Problem

Build a minimal Invoice & Payment Service, the kind of system that sits behind a billing product. A business creates invoices for their customers, customers pay invoices, and the business gets notified about state changes via webhooks.

The product surface is intentionally small. The interesting work is in the state machine, failure modes, and data model.

Domain Model (Required Shape)

```
Business --has many--> Customers
Business --has many--> Invoices --belongs to--> Customer
Invoice  --has many--> Payment Attempts
```

- A business authenticates with API keys.
- A customer belongs to one business (name plus email is enough).
- An invoice belongs to one customer, has line items, a total, a state, and a due date.
- A payment attempt records a try at paying an invoice (success, failure, or pending).

Money

- Single currency: **USD.**
- **Integer minor units (cents).** No floats anywhere in the money path. We will look for this.

Core Requirements (Must-Have)

1. **API key authentication.** Keys are scoped to a business. Storage, hashing, transmission, revocation: your call. Defend it in DESIGN.md.
2. **Customers.** Create, get, and list customers (scoped to the authenticated business).
3. **Invoices.** Create with line items {description, quantity, unit_amount_cents}. The server computes the total. Never trust a client-supplied total. Get by ID. List filterable by state. Each invoice has a state (you define the values).
4. **Payment attempts.** POST /invoices/{id}/pay with a mock card token. Records a payment attempt, calls the mock PSP, updates invoice state based on the result. Must be idempotent via Idempotency-Key header. Must handle the PSP being slow or failing without corrupting invoice state.
5. **Invoice state machine.** You define the states (typical: draft, open, paid, void, uncollectible). Required: a state diagram in DESIGN.md showing all valid transitions, what triggers each, and which states are terminal. Invalid transitions must be rejected at the API level with a clear error.
6. **Webhooks.** Businesses register endpoint URLs. They receive signed webhooks for at least invoice.created, invoice.paid, and invoice.payment_failed. Delivery must be signed (so receivers

can verify) and retried on failure with a documented backoff. Webhook delivery must not block the API response.

7. **PostgreSQL with migrations.**
8. **docker compose up.** Brings up app, database, and mock PSP with no further steps.
9. **README.** Run instructions and 3 to 4 curl examples covering: create customer, create invoice, attempt payment (success and failure cases).
10. **API documentation.** OpenAPI YAML or Markdown. Include request and response shapes, and a consistent error format.

Mock Payment Processor

We provide the spec. You build it.

Build a mock PSP service in the same repo (separate binary or a route prefix, your choice). The mock determines outcomes from a card token string in the request:

Token	Behavior
`tok_success`	Returns {status: "succeeded", psp_ref: <uuid>} after about 100 ms.
`tok_insufficient_funds`	Returns {status: "failed", code: "insufficient_funds"} after about 100 ms.
`tok_card_declined`	Returns {status: "failed", code: "card_declined"} after about 100 ms.
`tok_timeout`	**Sleeps 30 seconds then returns success.** Your service must handle this without hanging.
`tok_network_error`	Returns 500 or drops the connection. Your service must handle this.

Your invoice service calls the mock PSP over HTTP. Treat it as a real external dependency. How you handle `tok_timeout` and `tok_network_error` **is a key evaluation point.**

Explicitly Out of Scope

Do NOT build these.

- Subscriptions, recurring billing, plans, proration.
- Refunds or partial payments (mention them in DESIGN.md if you would add them later).
- Multi-currency or FX.
- Tax calculation.
- A frontend or UI.
- Email sending (logging “would send email” is fine).
- Production-grade rate limiting (discuss it in DESIGN.md instead).
- OAuth or any auth beyond API keys.

We value restraint

If you are tempted to build something not on the must-have list, write about it in DESIGN.md instead of building it. What you cut, and why, is a graded section.

The Design Document (DESIGN.md)

The main deliverable.

Code can be AI-generated. Design judgment cannot. This document is the primary thing we evaluate. Aim for roughly 800 to 1,500 words. Bullet points are fine. Diagrams welcome.

Address these sections explicitly:

1. Data Model

ER diagram or table listing. For each table: shape, indexes, primary key strategy, why this shape over alternatives, and what you would change at 100x scale.

2. Invoice State Machine

A diagram (ASCII, Mermaid, or image) showing all states, all valid transitions, the trigger for each transition, and which states are terminal. Then briefly: which transitions are reversible, and how invalid transitions are rejected.

3. Payment Correctness & Failure Modes

The hard section. Walk us through what happens in each case:

- **(a)** Two clients call POST /invoices/{id}/pay for the same invoice at the same instant. What is the outcome? What mechanism guarantees this?
- **(b)** The mock PSP times out (`tok_timeout`, 30 s). What does your endpoint return? What state is the invoice or payment_attempt left in? How does the caller find out the eventual result?
- **(c)** The PSP returns success but your service crashes before persisting that. What happens on retry? Does the customer get charged twice?
- **(d)** An idempotency key is reused with a *different* request body. What do you do?
- **(e)** An invoice in paid state receives another POST /pay. What happens?

Name your concurrency mechanism (row-level lock, advisory lock, optimistic concurrency, serializable isolation, status-conditional update, etc.) and explain why over the alternatives.

4. Webhook Design

Signing scheme (algorithm, what is signed, replay protection). Retry policy with **specific numbers** (intervals, max attempts, total budget). What happens to webhooks that exhaust the retry budget? How does a business reconcile missed events? Why is delivery decoupled from the API response path, and how?

5. API Key Model

Generation, storage (plaintext, hash, prefix?), transmission, rotation, revocation, and blast radius if leaked.

6. What You Cut and Why

List 3 to 5 things you deliberately did not build. This section is required.

7. Production Readiness Gap

If this shipped tomorrow, the top 3 things missing? (Observability, rate limiting, audit log, refunds, dunning, etc.) You do not need to build them. Just demonstrate you can see them.

Testing

We do not require a coverage number. We do require:

1. **One concurrency test** that fires N concurrent POST /pay requests for the same invoice and asserts that at most one succeeds, no double-charges occur, and the final state is consistent.
2. **One idempotency test** that retries the same request with the same key and asserts the same response is returned without a second PSP call.
3. **One PSP-failure test** that uses `tok_timeout` or `tok_network_error` and asserts the invoice is not stuck in a bad state.

Lean on these rather than testing every handler. If you skip any, explain why in the README.

Video Demo (Required)

Record a 5 to 10 minute screen recording (Loom, Zoom export, QuickTime, or any equivalent that produces a shareable link). The video is a required deliverable.

Your video must cover, in this order:

1. **Architecture overview (1 to 2 min).** Walk through your services, data model, and how a request flows from API to database to webhook delivery.
2. **Live demo (2 to 3 min).** Run `docker compose up` and demonstrate: create a customer, create an invoice, attempt a successful payment, attempt a failing payment (use `tok_card_declined`), and show the resulting webhook deliveries.
3. **State machine walkthrough (1 to 2 min, unscripted).** Explain your invoice state machine in your own words. Why these states? What transitions are allowed? What is terminal? Where did you deliberate?
4. **Failure-mode walkthrough (1 to 2 min, unscripted).** Pick ONE of the failure modes from DESIGN.md section 3 and explain, on camera, exactly how your code handles it. Open the relevant file and walk us through the lines. Use `tok_timeout` or `tok_network_error` if you want to demo it live.

> **Why we ask for the video**
>
> The unscripted walkthroughs of your state machine and one failure mode are how we verify ownership of the design. We are not looking for polish. We are looking for fluency. If you cannot explain a decision in your own words, you did not make it.

Submission format

- Include the video link in README.md under a section titled Demo Video.
- Loom links are fine. So are Google Drive, Dropbox, or S3 links. Just make sure they are accessible without a login.

- No editing required. Cuts, ums, and retries are fine. We want the working session, not a marketing reel.

Submission

A GitHub repository (public or shared with us) containing:

- **Source code**. Invoice service plus mock PSP.
- **Migrations**. SQL or migration framework files.
- **docker-compose.yml**. One-command setup.
- **README.md**. Run instructions, curl examples, and demo video link.
- **DESIGN.md**. Per the spec above. The primary deliverable.
- **AI_USAGE.md**. Per the spec above. Required.
- **API documentation**. OpenAPI YAML or Markdown.

How We Evaluate

We grade on four axes, weighted roughly equally:

Axis	What we look for
Design judgment *DESIGN.md, video walkthroughs*	Specific reasoning. The state machine is coherent and complete. Failure-mode answers are specific, not generic. You can explain your design verbally without reading from notes.
Core correctness	Money math is integer. Concurrent payments do not double-charge. Idempotency works. Invoice state transitions are valid. PSP failures do not corrupt state.
Operational sense	Migrations, consistent error format, sensible logging, clean docker compose up, webhooks decoupled from request path.
Communication *AI_USAGE.md, README, commits, video*	Honest, specific, shows what you contributed. Video walkthrough is fluent and unscripted.

What does NOT score points

- Lines of code.
- Number of features beyond the must-haves.
- Exhaustive test coverage.
- Fancy abstractions.
- Premature optimization.
- Dependencies you cannot justify.

What scores strongly

A small, correct, well-explained service where the design doc and video show how you think about state, money, and failure.

Quick Self-Check Before Submitting

docker compose up works on a clean machine, no manual steps.

No floats anywhere in the money path.

Concurrent-payment test exists and passes.

tok_timeout does not hang your endpoint.

DESIGN.md answers the failure-mode questions specifically (not generically).

State machine diagram is in DESIGN.md.

AI_USAGE.md is honest and specific.

Video link is in README.md and accessible without login.

Video covers all four required sections (architecture, demo, state machine, failure mode).

You cut at least one thing you wanted to build, and said why.

Questions about this assignment? Reach out before you start. We would rather clarify scope than have you guess. Good luck, and we look forward to seeing how you think.